```
# Mount Google Drive to access files stored in your Drive.
# This allows the notebook to read datasets or save outputs directl
from google.colab import drive

# Connects your Google Drive to the Colab environment at the folder
# After running this line, you will be asked to authorize access.
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remo
```

```
# Define the main project directory located inside Google Drive.
# This is the folder that contains all the datasets and project fil
base_path = "/content/drive/MyDrive/DeepFaceEmotion"

# Path to the FER2013 CSV file (contains pixel values and labels).
fer_csv_path = f"{base_path}/fer2013.csv"

# Path to the FER2013 ZIP file (used if you need to extract images
fer_zip_path = f"{base_path}/fer2013.zip"

# Path to the extracted RAF-DB dataset folder inside the project di
# This folder contains subfolders for each emotion category.
rafdb_output_path = f"{base_path}/RAF-DB"
```

```
import os

# Create the RAF-DB directory if it does not already exist.
# This ensures that the extracted files will have a destination fol
os.makedirs(rafdb_output_path, exist_ok=True)

# Extract the contents of the FER2013 ZIP file into the RAF-DB folc
# The "-o" flag forces overwrite if files already exist.
!unzip -o "{fer_zip_path}" -d "{rafdb_output_path}"
```

```
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/s
```

```
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
  inflating: /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/train/su
```

```
import pandas as pd

# Load the FER2013 dataset from the CSV file into a Pandas DataFram
# The CSV contains pixel values for each image and the correspondir
df_fer = pd.read_csv(fer_csv_path)

# Display the first five rows of the dataset to verify that it loac
df_fer.head()
```

	emotion	pixels	Usage
0	0	70 80 82 72 58 58 60 63 54 58 60 48 89 115 121...	Training
1	0	151 150 147 155 148 133 111 140 170 174 182 15...	Training
2	2	231 212 156 164 174 138 161 173 182 200 106 38...	Training
3	4	24 32 36 30 32 23 19 20 30 41 21 22 32 34 21 1...	Training
4	6	4 0 0 0 0 0 0 0 0 0 0 0 3 15 23 28 48 50 58 84...	Training

Next steps: Generate code with `df_fer` New interactive sheet

```
# List all files and folders inside the specified directory (recurs
# This is useful to verify that the RAF-DB dataset was extracted cc
!ls -R /content/rafa_db_extracted
```

```
ls: cannot access '/content/rafa_db_extracted': No such file or dire
```

```
# List all files and folders inside the main DeepFaceEmotion direct
# This helps verify that all required datasets and project files ar
!ls -R /content/DeepFaceEmotion
```

```
ls: cannot access '/content/DeepFaceEmotion': No such file or direct
```

```
# Recursively list all files and subdirectories inside the DeepFaceE
# This is useful to confirm that the project files and datasets exis
!ls -R /content/drive/MyDrive/DeepFaceEmotion
```

```
Training_38350171.jpg  Training_68251712.jpg  Training_98397319.jpg
Training_3841930.jpg   Training_6825582.jpg   Training_98409862.jpg
Training_38463972.jpg  Training_68266130.jpg  Training_98446939.jpg
Training_38491793.jpg  Training_68271960.jpg  Training_98447563.jpg
Training_38504835.jpg  Training_68277085.jpg  Training_98472640.jpg
Training_38553588.jpg  Training_68296400.jpg  Training_98481914.jpg
Training_38563383.jpg  Training_68316414.jpg  Training_98511054.jpg
Training_38588437.jpg  Training_68333910.jpg  Training_98514521.jpg
Training_38623456.jpg  Training_68347899.jpg  Training_98539906.jpg
Training_38629933.jpg  Training_68375521.jpg  Training_98569897.jpg
Training_38684227.jpg  Training_68404332.jpg  Training_98576352.jpg
Training_38697129.jpg  Training_68414946.jpg  Training_98588726.jpg
Training_387312.jpg    Training_68434675.jpg  Training_98594534.jpg
Training_38744859.jpg  Training_68480437.jpg  Training_98644924.jpg
Training_38749895.jpg  Training_68490945.jpg  Training_98657632.jpg
Training_38768082.jpg  Training_68497011.jpg  Training_98664620.jpg
Training_38791655.jpg  Training_68507681.jpg  Training_98737557.jpg
Training_38802577.jpg  Training_68554007.jpg  Training_98747340.jpg
Training_38943737.jpg  Training_68561658.jpg  Training_98750930.jpg
Training_3898286.jpg   Training_68618109.jpg  Training_98758885.jpg
Training_39050169.jpg  Training_68632316.jpg  Training_98767792.jpg
Training_39055823.jpg  Training_6866410.jpg   Training_98774734.jpg
Training_3908175.jpg   Training_68677403.jpg  Training_9880069.jpg
Training_39154431.jpg  Training_68730211.jpg  Training_98827007.jpg
Training_39191303.jpg  Training_68738656.jpg  Training_98828949.jpg
Training_39211513.jpg  Training_68745503.jpg  Training_98840937.jpg
Training_39214334.jpg  Training_68747987.jpg  Training_98861159.jpg
Training_39228942.jpg  Training_68787983.jpg  Training_98899707.jpg
Training_39238392.jpg  Training_68807103.jpg  Training_9890261.jpg
Training_39278480.jpg  Training_68817633.jpg  Training_98971238.jpg
Training_39280728.jpg  Training_68854449.jpg  Training_98972491.jpg
Training_39317714.jpg  Training_68860232.jpg  Training_99066709.jpg
Training_39433582.jpg  Training_68864353.jpg  Training_9909756.jpg
Training_39439878.jpg  Training_68867125.jpg  Training_99104204.jpg
Training_39441691.jpg  Training_68889399.jpg  Training_99106305.jpg
Training_39488337.jpg  Training_68976231.jpg  Training_99113691.jpg
Training_3951286.jpg   Training_68981394.jpg  Training_991508.jpg
Training_39536730.jpg  Training_68990617.jpg  Training_99182527.jpg
Training_39551863.jpg  Training_69082438.jpg  Training_99233432.jpg
Training_39572034.jpg  Training_69186969.jpg  Training_99267647.jpg
Training_39599333.jpg  Training_69199262.jpg  Training_99279802.jpg
Training_39605012.jpg  Training_69225397.jpg  Training_99318660.jpg
Training_39663636.jpg  Training_69232483.jpg  Training_9932811.jpg
Training_39671250.jpg  Training_69234558.jpg  Training_99399304.jpg
Training_39750918.jpg  Training_69284822.jpg  Training_99399823.jpg
Training_3976671.jpg   Training_69284843.jpg  Training_99404349.jpg
Training_3981392.jpg   Training_69303250.jpg  Training_9943398.jpg
Training_39825422.jpg  Training_69316941.jpg  Training_99441101.jpg
Training_39862056.jpg  Training_69351834.jpg  Training_99576616.jpg
Training_39877500.jpg  Training_694448.jpg    Training_99604118.jpg
Training_39913247.jpg  Training_69528483.jpg  Training_99663019.jpg
Training_39930861.jpg  Training_69540102.jpg  Training_99791966.jpg
Training_3993891.jpg   Training_69582464.jpg  Training_99794165.jpg
```

```
Training_39944091.jpg  Training_6960441.jpg   Training_99807705.jp
Training_39959405.jpg  Training_69615552.jpg  Training_99827442.jp
Training_39959891.jpg  Training_69617945.jpg  Training_99916297.jp
Training_40002746.jpg  Training_6966074.jpg   Training_99924420.jp
```

```
# Recursively list all files and subfolders inside the RAF-DB direc
# This allows you to verify that the RAF-DB dataset was extracted c
!ls -R /content/drive/MyDrive/DeepFaceEmotion/RAF-DB
```

```
Training_38332922.jpg  Training_68236223.jpg  Training_9839279.jpg
Training_38349249.jpg  Training_68236333.jpg  Training_98393554.jp
Training_38350171.jpg  Training_68251712.jpg  Training_98397319.jp
Training_3841930.jpg   Training_6825582.jpg   Training_98409862.jp
Training_38463972.jpg  Training_68266130.jpg  Training_98446939.jp
Training_38491793.jpg  Training_68271960.jpg  Training_98447563.jp
Training_38504835.jpg  Training_68277085.jpg  Training_98472640.jp
Training_38553588.jpg  Training_68296400.jpg  Training_98481914.jp
Training_38563383.jpg  Training_68316414.jpg  Training_98511054.jp
Training_38588437.jpg  Training_68333910.jpg  Training_98514521.jp
Training_38623456.jpg  Training_68347899.jpg  Training_98539906.jp
Training_38629933.jpg  Training_68375521.jpg  Training_98569897.jp
Training_38684227.jpg  Training_68404332.jpg  Training_98576352.jp
Training_38697129.jpg  Training_68414946.jpg  Training_98588726.jp
Training_387312.jpg    Training_68434675.jpg  Training_98594534.jp
Training_38744859.jpg  Training_68480437.jpg  Training_98644924.jp
Training_38749895.jpg  Training_68490945.jpg  Training_98657632.jp
Training_38768082.jpg  Training_68497011.jpg  Training_98664620.jp
Training_38791655.jpg  Training_68507681.jpg  Training_98737557.jp
Training_38802577.jpg  Training_68554007.jpg  Training_98747340.jp
Training_38943737.jpg  Training_68561658.jpg  Training_98750930.jp
Training_3898286.jpg   Training_68618109.jpg  Training_98758885.jp
Training_39050169.jpg  Training_68632316.jpg  Training_98767792.jp
Training_39055823.jpg  Training_6866410.jpg   Training_98774734.jp
Training_3908175.jpg   Training_68677403.jpg  Training_9880069.jpg
Training_39154431.jpg  Training_68730211.jpg  Training_98827007.jp
Training_39191303.jpg  Training_68738656.jpg  Training_98828949.jp
Training_39211513.jpg  Training_68745503.jpg  Training_98840937.jp
Training_39214334.jpg  Training_68747987.jpg  Training_98861159.jp
Training_39228942.jpg  Training_68787983.jpg  Training_98899707.jp
Training_39238392.jpg  Training_68807103.jpg  Training_9890261.jpg
Training_39278480.jpg  Training_68817633.jpg  Training_98971238.jp
Training_39280728.jpg  Training_68854449.jpg  Training_98972491.jp
Training_39317714.jpg  Training_68860232.jpg  Training_99066709.jp
Training_39433582.jpg  Training_68864353.jpg  Training_9909756.jpg
Training_39439878.jpg  Training_68867125.jpg  Training_99104204.jp
Training_39441691.jpg  Training_68889399.jpg  Training_99106305.jp
Training_39488337.jpg  Training_68976231.jpg  Training_99113691.jp
Training_3951286.jpg   Training_68981394.jpg  Training_991508.jpg
Training_39536730.jpg  Training_68990617.jpg  Training_99182527.jp
```

```
Training_39551863.jpg  Training_69082438.jpg  Training_99233432.jp
Training_39572034.jpg  Training_69186969.jpg  Training_99267647.jp
Training_39599333.jpg  Training_69199262.jpg  Training_99279802.jp
Training_39605012.jpg  Training_69225397.jpg  Training_99318660.jp
Training_39663636.jpg  Training_69232483.jpg  Training_9932811.jpg
Training_39671250.jpg  Training_69234558.jpg  Training_99399304.jp
Training_39750918.jpg  Training_69284822.jpg  Training_99399823.jp
Training_3976671.jpg   Training_69284843.jpg  Training_99404349.jp
Training_3981392.jpg   Training_69303250.jpg  Training_9943398.jpg
Training_39825422.jpg  Training_69316941.jpg  Training_99441101.jp
Training_39862056.jpg  Training_69351834.jpg  Training_99576616.jp
Training_39877500.jpg  Training_694448.jpg    Training_99604118.jp
Training_39913247.jpg  Training_69528483.jpg  Training_99663019.jp
Training_39930861.jpg  Training_69540102.jpg  Training_99791966.jp
Training_3993891.jpg   Training_69582464.jpg  Training_99794165.jp
Training_39944091.jpg  Training_6960441.jpg   Training_99807705.jp
Training_39959405.jpg  Training_69615552.jpg  Training_99827442.jp
Training_39959891.jpg  Training_69617945.jpg  Training_99916297.jp
```

```
# List the first few items inside the RAF-DB test directory.
# Using 'head' limits the output, which is helpful when the folder
!ls /content/drive/MyDrive/DeepFaceEmotion/RAF-DB/test | head
```

```
angry
disgust
fear
happy
neutral
sad
surprise
```

```
# List all items inside the RAF-DB directory (non-recursive).
# This provides a quick overview of the main folders and files in t
!ls /content/drive/MyDrive/DeepFaceEmotion/RAF-DB
```

```
test  train
```

```
# INSTALL OPENCV FIRST
!pip install opencv-python-headless

# ============================================
# SETUP & IMPORTS
# ============================================

from google.colab import drive
drive.mount('/content/drive')

import os
import numpy as np
import pandas as pd
import glob
import cv2
import matplotlib.pyplot as plt
from tqdm import tqdm
from PIL import Image

from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D,
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPla
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2
from tensorflow.keras.applications import VGG19, ResNet50

import json
```

```
Requirement already satisfied: opencv-python-headless in /usr/local/
Requirement already satisfied: numpy<2.3.0,>=2 in /usr/local/lib/pyt
Drive already mounted at /content/drive; to attempt to forcibly remo
```

```
# Base folder in Google Drive where all project files and datasets
base_path = "/content/drive/MyDrive/DeepFaceEmotion"

# Full path to the FER2013 CSV file containing pixel values and emo
fer_csv_path = f"{base_path}/fer2013.csv"

# Paths to the RAF-DB training and testing folders.
# These directories contain real-world facial images organized by e
raf_train_path = f"{base_path}/RAF-DB/train"
raf_test_path = f"{base_path}/RAF-DB/test"
```

```
# Load the FER2013 dataset from the CSV file into a DataFrame.
# The CSV contains pixel values stored as a long space-separated st
df_fer = pd.read_csv(fer_csv_path)

# Function to convert the pixel string from each row into an actual
# Each image is reshaped to 48x48 pixels (grayscale) and saved as a
def pixels_to_image(row, save_dir="/content/fer_images"):
    os.makedirs(save_dir, exist_ok=True)  # Create the folder if it
    img_array = np.array(row["pixels"].split(), dtype=np.uint8).res
    img_path = os.path.join(save_dir, f"fer_{row.name}.jpg")
    Image.fromarray(img_array).convert("RGB").save(img_path)
    return img_path

# Convert all FER2013 pixel rows into actual image files and store
df_fer["image_path"] = df_fer.apply(pixels_to_image, axis=1)

# Keep only the image path and label columns, and rename 'emotion'
df_fer = df_fer[["image_path", "emotion"]].rename(columns={"emotion

# Add a column to indicate that these samples came from the FER2013
df_fer["dataset"] = "FER2013"

print("✔ FER2013 images generated successfully")
```

```
✔ FER2013 images generated successfully
```

```
# Lists to store the file paths and labels for RAF-DB training imag
raf_train_imgs = []
raf_train_labels = []

# Loop through each subfolder inside the RAF-DB train directory.
# Each subfolder represents an emotion category.
for folder in os.listdir(raf_train_path):
    class_path = os.path.join(raf_train_path, folder)
    if os.path.isdir(class_path):

        # Collect all JPG images inside the emotion folder.
        for img in glob.glob(os.path.join(class_path, "*.jpg")):
            raf_train_imgs.append(img)
            raf_train_labels.append(folder)

# Create a DataFrame with image paths and their corresponding label
df_raf = pd.DataFrame({"image_path": raf_train_imgs, "label": raf_t
df_raf["dataset"] = "RAF-DB"  # Mark dataset origin

# Encode text labels (e.g., 'happy', 'angry') into numeric class ID
unique_labels = sorted(df_raf["label"].unique())
label_to_num = {label: idx for idx, label in enumerate(unique_label
df_raf["label"] = df_raf["label"].map(label_to_num).astype(int)

print("✔ RAF-DB train loaded successfully")
```

```
✔ RAF-DB train loaded successfully
```

```
df_combined = pd.concat([df_fer, df_raf], ignore_index=True)
df_combined["label"] = df_combined["label"].astype(int)

# Split into train/val/test
train_df, test_df = train_test_split(df_combined, test_size=0.15, s
train_df, val_df = train_test_split(train_df, test_size=0.15, strat

# Convert labels to string for the generator
train_df["label"] = train_df["label"].astype(str)
val_df["label"] = val_df["label"].astype(str)

print("✔ Dataset combined and split successfully")
```

```
✔ Dataset combined and split successfully
```

```
# Target image size for all input samples.
# All images will be resized to 96x96 pixels before being fed into
target_img_size = (96, 96)

# Data augmentation for the training set.
# These transformations help increase dataset variability and reduc
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

# Only rescaling for the validation set (no augmentation applied).
val_datagen = ImageDataGenerator(rescale=1./255)

# Create the training generator from the DataFrame.
# Images are read directly from disk using their file paths.
train_generator = train_datagen.flow_from_dataframe(
    train_df,
    x_col="image_path",
    y_col="label",
    target_size=target_img_size,
    batch_size=32,
    class_mode="categorical",
    shuffle=True
)

# Create the validation generator (no shuffling for consistent eval
val_generator = val_datagen.flow_from_dataframe(
    val_df,
    x_col="image_path",
    y_col="label",
    target_size=target_img_size,
    batch_size=32,
    class_mode="categorical",
    shuffle=False
)
```

```
Found 46670 validated image filenames belonging to 7 classes.
Found 8236 validated image filenames belonging to 7 classes.
```

```
# Load the EfficientNetB0 model pre-trained on ImageNet as the feat
# include_top=False removes the original classification layers so w
# input_shape defines the size of the resized images (96x96 RGB).
from tensorflow.keras.applications import EfficientNetB0
base_model = EfficientNetB0(weights='imagenet', include_top=False,
```

```
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2

# ============================================================
#   LOAD EfficientNetB0
# ============================================================
base_model = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(96, 96, 3)
)

# ============================================================
#   PARTIAL FINE-TUNING (IMPORTANT!!)
#   - Freeze first ~80% of layers
#   - Unfreeze last 50 layers
# ============================================================

# Step A: Freeze all layers first
for layer in base_model.layers:
    layer.trainable = False

# Step B: Unfreeze the last 80 layers
for layer in base_model.layers[-80:]:
    layer.trainable = True

print("EfficientNetB0 partially unfrozen — last 50 layers are traina

# ============================================================
#   ADD CUSTOM CLASSIFICATION HEAD
# ============================================================
x = base_model.output
x = GlobalAveragePooling2D()(x)

x = Dense(256, activation='relu', kernel_regularizer=l2(0.02))(x)
x = BatchNormalization()(x)
```

```
x = Dropout(0.5)(x)

x = Dense(128, activation='relu', kernel_regularizer=l2(0.02))(x)
x = Dropout(0.3)(x)

outputs = Dense(len(train_generator.class_indices), activation='soft

model = Model(inputs=base_model.input, outputs=outputs)

# ============================================================
#   COMPILE WITH LOWER LR (CRITICAL FOR FINE-TUNING)
# ============================================================
model.compile(
    optimizer=Adam(learning_rate=1e-5),   # <<<
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("✔ Model compiled with Fine-Tuning (LR=1e-5)")
```

```
EfficientNetB0 partially unfrozen — last 50 layers are trainable!
✔ Model compiled with Fine-Tuning (LR=1e-5)
```

```
# ============================================================
#   TRAINING THE EFFICIENTNETB0 (AFTER FIXING FREEZE ISSUE)
# ============================================================

# 1) Class Weights (keep as is)
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

classes = np.unique(train_df["label"].astype(int))
weights = compute_class_weight(
    class_weight='balanced',
    classes=classes,
    y=train_df["label"].astype(int)
)
class_weights = dict(enumerate(weights))

# 2) Callbacks (keep as is)
callbacks = [
    EarlyStopping(monitor='val_accuracy', patience=12, restore_best_
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=4),
]

# 3) TRAIN THE MODEL (UPDATED EPOCHS)
history = model.fit(
```

```
    train_generator,
    validation_data=val_generator,
    epochs=25,                    # <<<
    callbacks=callbacks,
    class_weight=class_weights,   # <<<
    verbose=1
)

print("Training Completed Successfully!")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adap
  self._warn_if_super_not_called()
Epoch 1/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 241s 139ms/step - accuracy: 0.1231 -
Epoch 2/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 167s 115ms/step - accuracy: 0.1365 -
Epoch 3/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 167s 114ms/step - accuracy: 0.1391 -
Epoch 4/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1458 -
Epoch 5/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 169s 116ms/step - accuracy: 0.1444 -
Epoch 6/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1487 -
Epoch 7/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 165s 113ms/step - accuracy: 0.1472 -
Epoch 8/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1482 -
Epoch 9/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 167s 114ms/step - accuracy: 0.1534 -
Epoch 10/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1545 -
Epoch 11/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1568 -
Epoch 12/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 168s 115ms/step - accuracy: 0.1566 -
Epoch 13/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 168s 115ms/step - accuracy: 0.1578 -
Epoch 14/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1578 -
Epoch 15/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1632 -
Epoch 16/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1611 -
Epoch 17/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 167s 115ms/step - accuracy: 0.1680 -
Epoch 18/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1722 -
Epoch 19/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1750 -
Epoch 20/25
```

```
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1802 -
Epoch 21/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 114ms/step - accuracy: 0.1723 -
Epoch 22/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 166s 113ms/step - accuracy: 0.1719 -
Epoch 23/25
1459/1459 ━━━━━━━━━━━━━━━━━━━━ 164s 113ms/step - accuracy: 0.1732 -
Training Completed Successfully!
```

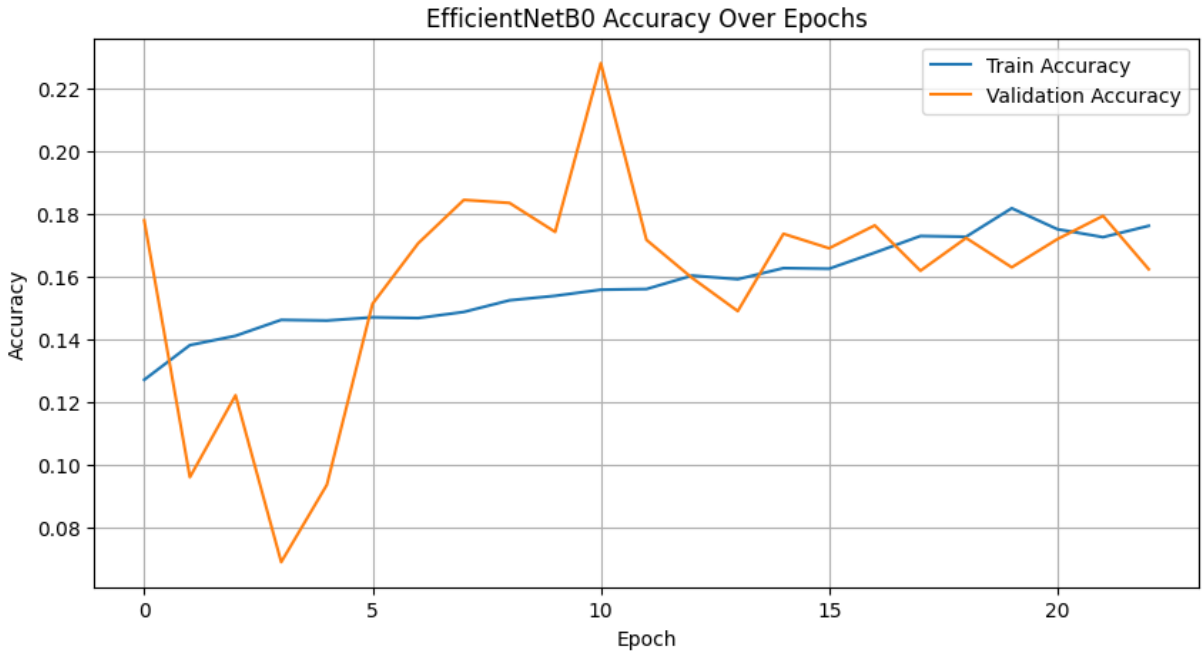
```
# ------------------------------------------------------------
# Reset the validation generator to ensure predictions
# start from the first batch and maintain correct ordering.
# ------------------------------------------------------------
val_generator.reset()

# ------------------------------------------------------------
# Predict probability scores for each image in the validation set
# using the EfficientNetB0 model.
# ------------------------------------------------------------
y_pred_probs = model.predict(val_generator)

# ------------------------------------------------------------
# Convert predicted probability vectors into final class labels
# by selecting the index of the maximum probability.
# ------------------------------------------------------------
y_pred = np.argmax(y_pred_probs, axis=1)

# ------------------------------------------------------------
# Extract the true class labels from the validation generator.
# This is used to compare predictions vs. actual labels.
# ------------------------------------------------------------
y_true = val_generator.classes

print("✔ Predictions for EfficientNet model completed.")
```

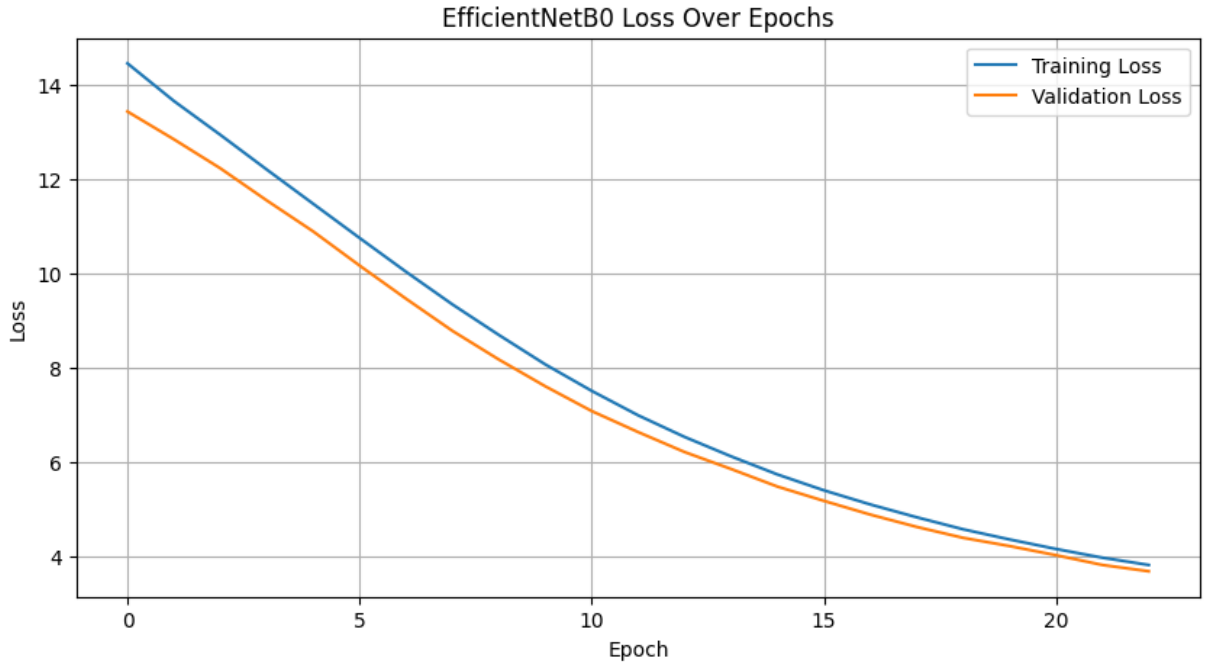
```
258/258 ━━━━━━━━━━━━━━━━━━━━ 24s 66ms/step
✔ Predictions for EfficientNet model completed.
```

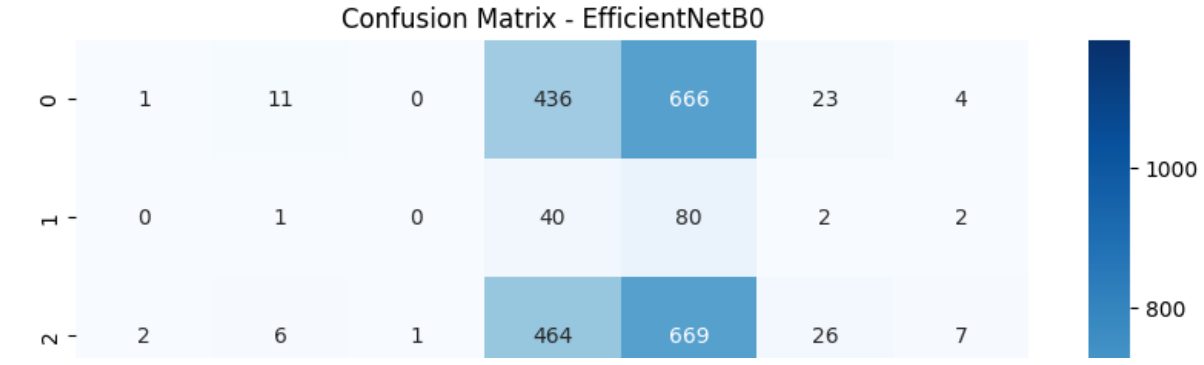
```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

# ============================================================
# 1) Plot Training & Validation Accuracy (EfficientNetB0)
# ============================================================
```

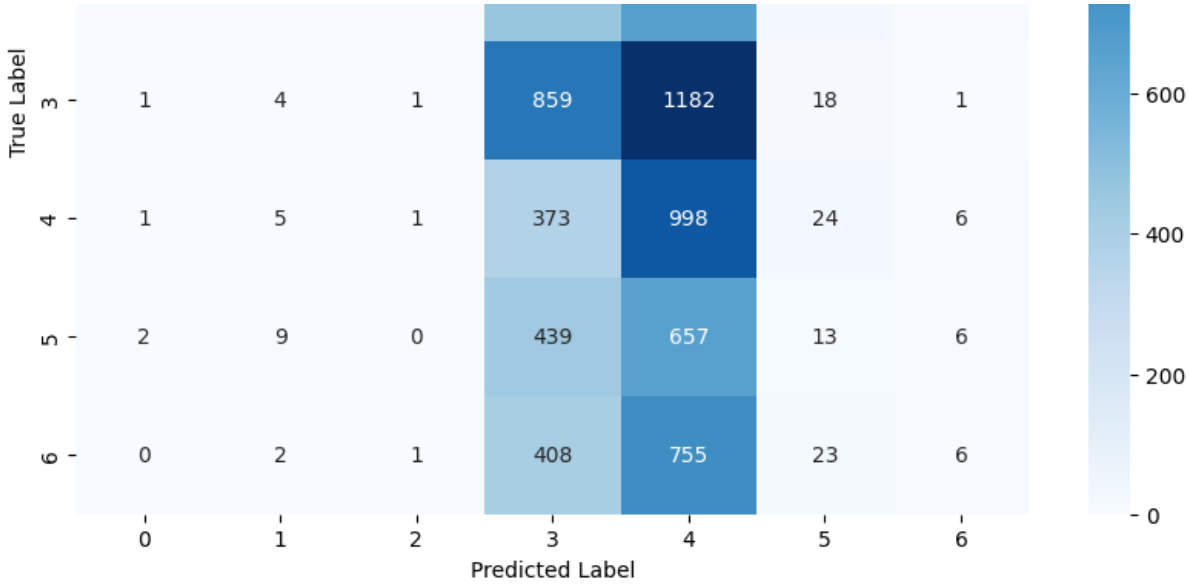
```
plt.figure(figsize=(10,5))

# Plot accuracy curves from the EfficientNet training history
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accurac

plt.title("EfficientNetB0 Accuracy Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# ============================================================
# 2) Plot Training & Validation Loss (EfficientNetB0)
# ============================================================
plt.figure(figsize=(10,5))

# Plot loss curves from the EfficientNet training history
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("EfficientNetB0 Loss Over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

# ============================================================
# 3) Confusion Matrix (EfficientNetB0)
# ============================================================

# Compute confusion matrix using true vs. predicted labels
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10,7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title("Confusion Matrix - EfficientNetB0")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

# ============================================================
# 4) Classification Report (EfficientNetB0)
# ============================================================
print("\n===== EfficientNetB0 Classification Report =====\n")
```

```
# Show detailed precision, recall, F1-score for each class
print(classification_report(y_true, y_pred))
```










```
===== EfficientNetB0 Classification Report =====

              precision    recall  f1-score   support

           0       0.14      0.00      0.00      1141
           1       0.03      0.01      0.01       125
           2       0.25      0.00      0.00      1175
           3       0.28      0.42      0.34      2066
           4       0.20      0.71      0.31      1408
           5       0.10      0.01      0.02      1126
           6       0.19      0.01      0.01      1195

    accuracy                           0.23      8236
   macro avg       0.17      0.16      0.10      8236
weighted avg       0.20      0.23      0.14      8236
```

```
# Convert test labels to string (required for categorical mode)
test_df["label"] = test_df["label"].astype(str)

# Create test generator
test_generator = val_datagen.flow_from_dataframe(
    test_df,
    x_col="image_path",
    y_col="label",
    target_size=(96, 96),
    batch_size=32,
    class_mode="categorical",
    shuffle=False
)
```

```
Found 9690 validated image filenames belonging to 7 classes.
```

```
test_loss, test_acc = model.evaluate(test_generator)
print(test_acc, test_loss)
```

```
  1/303 ━━━━━━━━━━━━━━━━━━━━ 22s 73ms/step - accuracy: 0.2812 - loss
  self._warn_if_super_not_called()
303/303 ━━━━━━━━━━━━━━━━━━━━ 18s 58ms/step - accuracy: 0.2295 - loss
0.22518059611320496 7.079372406005859
```

```
# ============================================================
#   FULL MODEL EVALUATION BLOCK (EfficientNetB0)
#   Includes:
#   - Test Accuracy & Loss
#   - Confusion Matrix (Raw + Normalized)
#   - Per-class Accuracy
#   - Classification Report
#   - ROC-AUC (Multi-class)
#   - Misclassified Images Visualization
#   - Model Parameter Count
#   - Learning Rate Curve
#   - Training Time Per Epoch
```

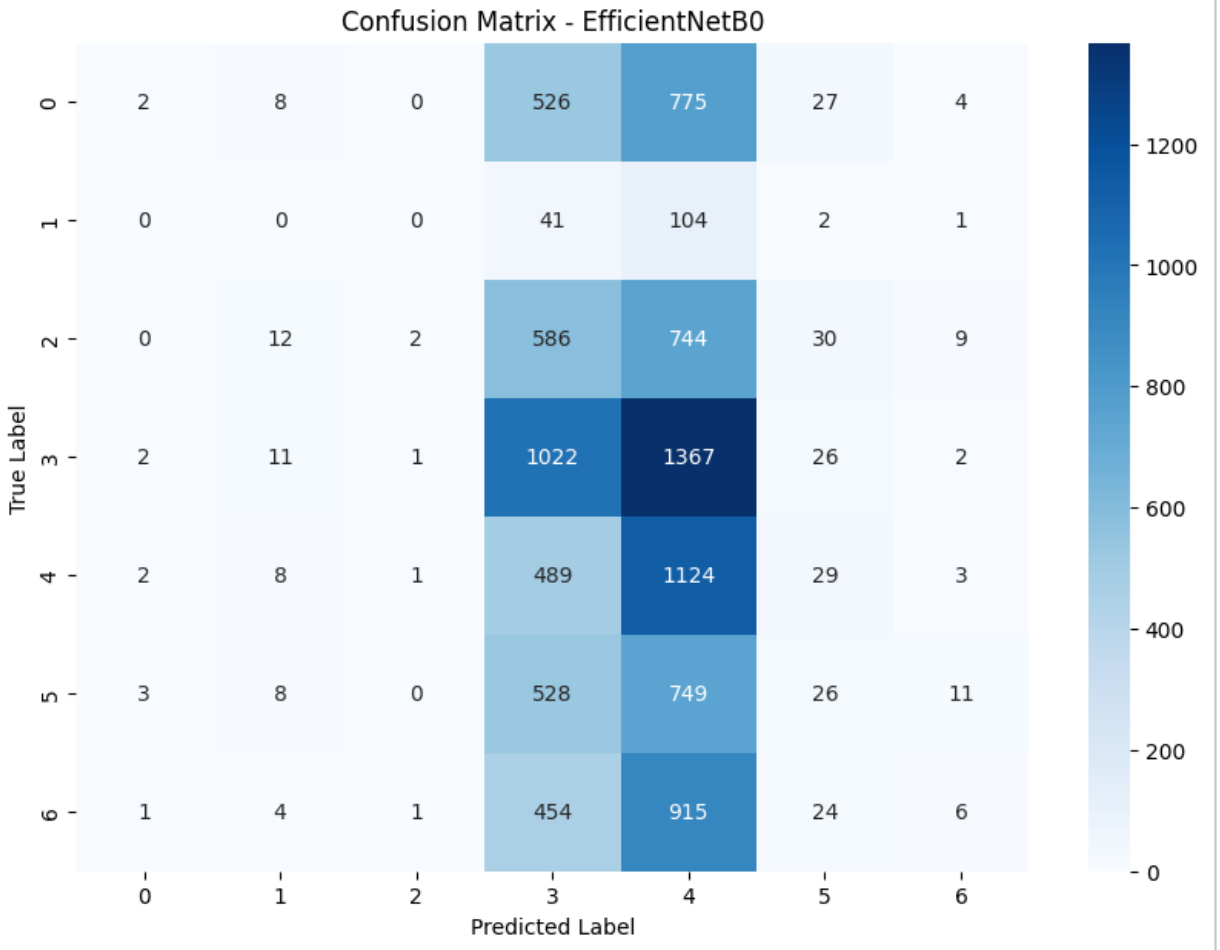
```
#   - Training Time Per Epoch
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report,
import time
import tensorflow as tf

# ------------------------------------------------------------
# 1) TEST SET EVALUATION
# ------------------------------------------------------------
print("=== Test Set Evaluation ===")

test_df["label"] = test_df["label"].astype(str)

test_generator = val_datagen.flow_from_dataframe(
    test_df,
    x_col="image_path",
    y_col="label",
    target_size=(96, 96),
    batch_size=32,
    class_mode="categorical",
    shuffle=False
)

test_loss, test_acc = model.evaluate(test_generator)
print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test Loss: {test_loss:.4f}")

# ------------------------------------------------------------
# 2) CONFUSION MATRIX (RAW + NORMALIZED)
# ------------------------------------------------------------
print("\n=== Confusion Matrix ===")

y_true_test = test_generator.classes
y_pred_test_probs = model.predict(test_generator)
y_pred_test = np.argmax(y_pred_test_probs, axis=1)

cm = confusion_matrix(y_true_test, y_pred_test)

plt.figure(figsize=(10,7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - EfficientNetB0")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

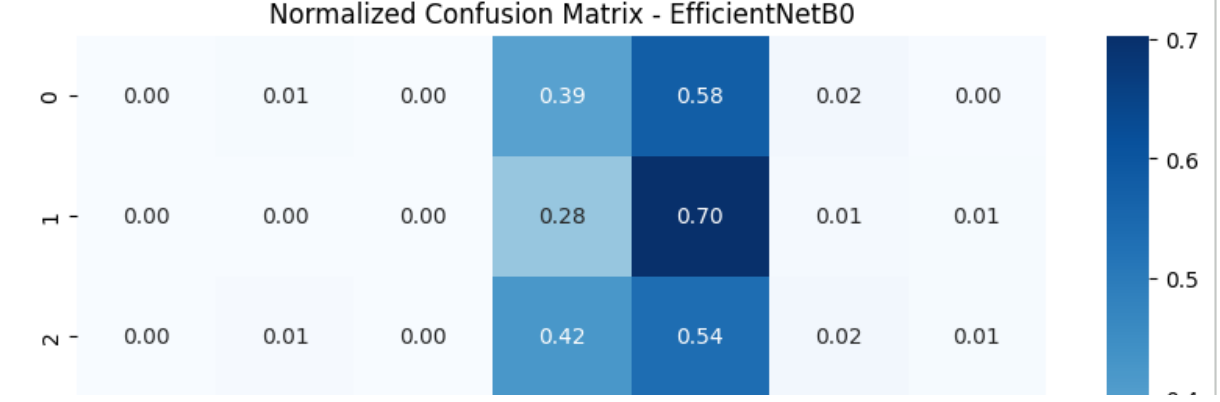
```
# Normalized version
cm_norm = cm.astype("float") / cm.sum(axis=1)[:, np.newaxis]

plt.figure(figsize=(10,7))
sns.heatmap(cm_norm, annot=True, cmap='Blues', fmt=".2f")
plt.title("Normalized Confusion Matrix - EfficientNetB0")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

# ------------------------------------------------------------
# 3) PER-CLASS ACCURACY
# ------------------------------------------------------------
print("\n=== Per-Class Accuracy ===")
per_class_acc = cm.diagonal() / cm.sum(axis=1)
for i, acc in enumerate(per_class_acc):
    print(f"Class {i} Accuracy: {acc:.2f}")

# ------------------------------------------------------------
# 4) CLASSIFICATION REPORT
# ------------------------------------------------------------
print("\n=== Classification Report ===")
print(classification_report(y_true_test, y_pred_test))

# ------------------------------------------------------------
# 5) ROC-AUC (MACRO) FOR MULTI-CLASS
# ------------------------------------------------------------
print("\n=== ROC-AUC (Macro) ===")

y_true_onehot = tf.keras.utils.to_categorical(
    y_true_test,
    num_classes=len(train_generator.class_indices)
)

auc_score = roc_auc_score(
    y_true_onehot,
    y_pred_test_probs,
    multi_class="ovo",
    average="macro"
)

print(f"Macro ROC-AUC: {auc_score:.4f}")

# ------------------------------------------------------------
# 6) MISCLASSIFIED IMAGE EXAMPLES
# ------------------------------------------------------------
print("\n=== Misclassified Images ===")
```

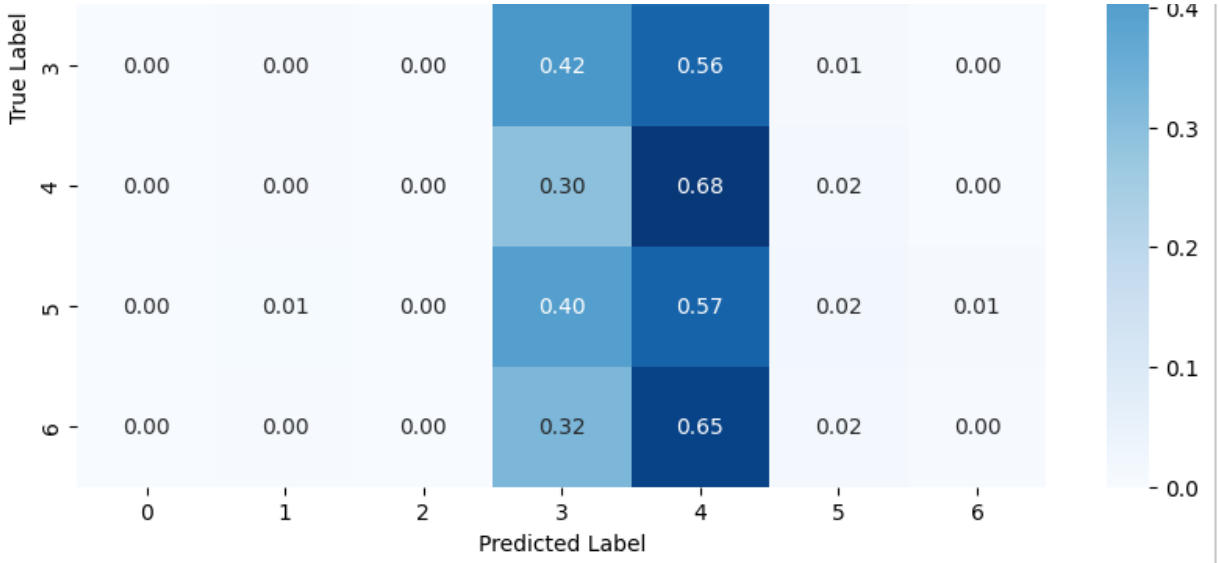
```
mis_idx = np.where(y_pred_test != y_true_test)[0][:12]

plt.figure(figsize=(12,8))
for i, idx in enumerate(mis_idx):
    img = plt.imread(test_generator.filepaths[idx])
    plt.subplot(3,4,i+1)
    plt.imshow(img)
    plt.title(f"True: {y_true_test[idx]} | Pred: {y_pred_test[idx]}"
    plt.axis('off')

plt.suptitle("Sample Misclassified Images")
plt.show()

# ------------------------------------------------------------
# 7) MODEL SIZE / PARAMETERS
# ------------------------------------------------------------
print("\n=== Model Parameter Count ===")
total_params = model.count_params()
print(f"Total Trainable & Non-Trainable Parameters: {total_params:,}

# ------------------------------------------------------------
# 8) LEARNING RATE CURVE (IF AVAILABLE)
# ------------------------------------------------------------
print("\n=== Learning Rate Curve (if tracked) ===")

if "lr" in history.history:
    plt.figure(figsize=(10,5))
    plt.plot(history.history["lr"])
    plt.title("Learning Rate Over Epochs")
    plt.xlabel("Epoch")
    plt.ylabel("Learning Rate")
    plt.grid(True)
    plt.show()
else:
    print("Learning rate not tracked in this training session.")

# ------------------------------------------------------------
# 9) TRAINING TIME PER EPOCH (APPROX)
# ------------------------------------------------------------
print("\n=== Approximate Training Time per Epoch ===")

start = time.time()
model.fit(
    train_generator,
    epochs=1,
    steps_per_epoch=1,
    verbose=0
)
```

```
end = time.time()

print(f"Approx. Time per Epoch: {end - start:.2f} seconds")
print("\n=== FULL ANALYSIS COMPLETED ===")
```

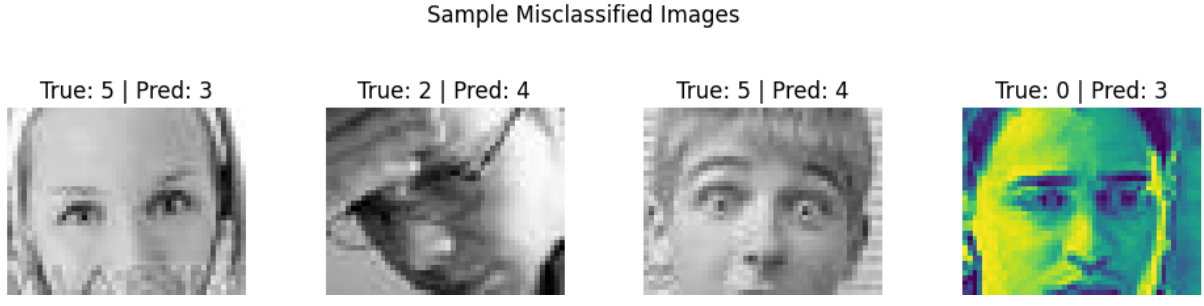
```
=== Test Set Evaluation ===
Found 9690 validated image filenames belonging to 7 classes.
  1/303 ━━━━━━━━━━━━━━━━━━━━ 22s 74ms/step - accuracy: 0.2812 - loss
  self._warn_if_super_not_called()
303/303 ━━━━━━━━━━━━━━━━━━━━ 12s 40ms/step - accuracy: 0.2295 - loss
Test Accuracy: 0.2252
Test Loss: 7.0794

=== Confusion Matrix ===
303/303 ━━━━━━━━━━━━━━━━━━━━ 17s 55ms/step
```

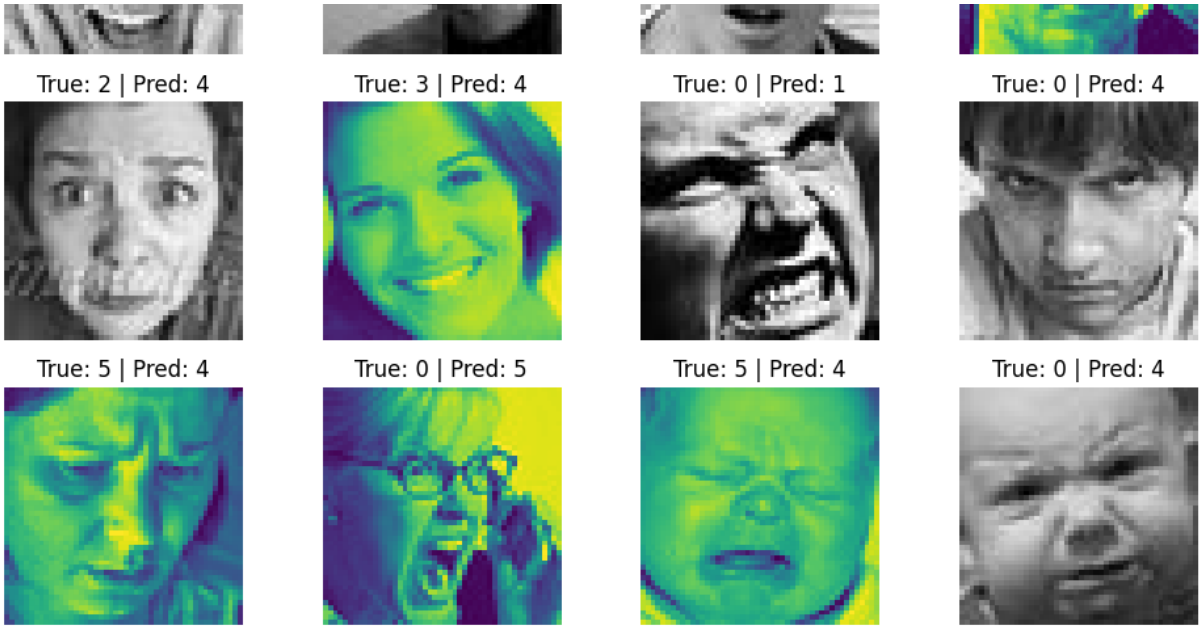







```
=== Per-Class Accuracy ===
Class 0 Accuracy: 0.00
Class 1 Accuracy: 0.00
Class 2 Accuracy: 0.00
Class 3 Accuracy: 0.42
Class 4 Accuracy: 0.68
Class 5 Accuracy: 0.02
Class 6 Accuracy: 0.00

=== Classification Report ===
              precision    recall  f1-score   support

           0       0.20      0.00      0.00      1342
           1       0.00      0.00      0.00       148
           2       0.40      0.00      0.00      1383
           3       0.28      0.42      0.34      2431
           4       0.19      0.68      0.30      1656
           5       0.16      0.02      0.03      1325
           6       0.17      0.00      0.01      1405

    accuracy                           0.23      9690
   macro avg       0.20      0.16      0.10      9690
weighted avg       0.23      0.23      0.14      9690


=== ROC-AUC (Macro) ===
Macro ROC-AUC: 0.5319

=== Misclassified Images ===
```






```
=== Model Parameter Count ===
Total Trainable & Non-Trainable Parameters: 4,412,330

=== Learning Rate Curve (if tracked) ===
Learning rate not tracked in this training session.

=== Approximate Training Time per Epoch ===
```